Software Testing and Quality Assurance

Course Code: **CS453**

Dr. Manar Elkady

Assignment 1: Testing Report

Name	ID
Shahd Mohamed Ahmed	20220533
Malak Hisham	20221162

Test Design Methodology:

1. **Input Space Partitioning (ISP)**

   We divided inputs into:

   - Valid inputs (normal cases)
   - Invalid inputs (null, negative, empty, exist, non-exist)
   - Edge cases (boundary conditions, large inputs, Empty collections)

2. **Boundary Value Analysis (BVA)**

   We focused on critical boundaries such as:

   - n = 0, 1, >1
   - Equal length vs shorter/longer

3. **Coverage Criteria**

   We ensured:

   - Statement Coverage
   - Branch Coverage (loops executed and skipped)
   - Condition Coverage (**if conditions (true/false)** are tested)

<u>SECTION 1: StringUtil Testing:</u>

1. Function: repeat (String s, int n):

Parameter	Partitions (ISP)	BVA
s	null, empty, normal, unicode	-1, 0, 1, large
n	negative, zero, positive, large	null, "

Test Cases:

Test Case	Input	Expected Output	Type
testRepeatNormal()	("s", 4)	"ssss"	ISP: Normal
testRepeatPositive()	("a", 1)	"a"	BVA: Positive
testRepeatZero()	("a", 0)	null	BVA: Zero
testRepeatNegative()	("a", -1)	null	BVA: negative
repeat_largeN()	("a", 1000)	length = 1000	Large Number
testRepeatNullString()	(null, 4)	null	ISP: Null
testRepeatEmptyString()	("", 6)	""	ISP: Empty
repeat_unicode()	("🔥", 2)	"🔥🔥"	ISP: Unicode

Coverage All branches of:

- null check
- n <= 0
- loop execution

2. Function: formatLeft(String s, String mask)

Parameter	Partitions (ISP)	BVA
s	null, empty, shorter, equal, longer	length: 0, equal, greater
mask	null, empty, shorter, equal, longer	length: 0, equal, greater

Test Cases:

Test Case	Input	Expected Output	Type
formatLeft_normal()	("abc", "123456")	"abc456"	ISP: normal
formatLeft_equalLength()	("abc", "123")	"abc"	BVA: equal length
formatLeft_stringLonger()	("abcdef", "123")	"abcdef"	BVA: greater length
formatLeft_nullString()	(null, "123")	"123"	ISP: null S

Test Case	Input	Expected Output	Type
formatLeft_nullMask()	("abc", null)	"abc"	ISP: null mask
formatLeft_emptyString()	("", "123")	"123"	ISP: empty S
formatLeft_emptyMask()	("abc", "")	"abc"	ISP: empty mask

Coverage All branches:

- null handling
- length comparison

3. Function: formatLeft(String s, String mask)

Parameter	Partitions (ISP)	BVA
s	null, empty, shorter, equal, longer	length: 0, equal, greater
mask	null, empty, shorter, equal, longer	length: 0, equal, greater

Test Cases:

Test Case	Input	Expected Output	Type
formatRight_normal()	("abc", "123456")	"123abc"	ISP: normal
formatRight_boundary_equalLength()	("abc", "123")	"abc"	BVA: equal length
formatRight_stringLonger()	("abcdef", "123")	"abcdef"	BVA: greater length
formatRight_nullString()	(null, "123")	"123"	ISP: null S
formatRight_nullMask()	("abc", null)	"abc"	ISP: null mask
formatRight_emptyString()	("", "123")	"123"	ISP: empty S
formatRight_emptyMask()	("abc", "")	"abc"	ISP: empty mask

Coverage All branches:

- null handling
- length comparison

SECTION 2: Multiset_HashMapImpl Testing:

1. **Function: add (T entry)**

Parameter	Partitions (ISP)	BVA
entry	New element, Existing element, Multiple insertions, Null key	0 → 1 → 2 null

Test Cases:

Test Case	Input	Expected Output	Comments
add_newElement	Add "a" to empty multiset	Old count = 0, New count = 1	ISP: new element
add_existingElement	Add "a" twice	Old count = 1, New count = 2	ISP: existing element
add_multipleTimes	Add "a" 3 times	Final count = 3	ISP: multiple insertions
add_nullKey	Add null	Count for null = 1	BVA: null key boundary

Coverage:

- All branches covered
- Null handling covered
- Boundary transitions covered

2. Function: add (T entry, int mult)

Partitions (ISP)	BVA
positive mult zero, negative, null key	mult = 0, 1, >1

Test Cases:

Test Case	Input	Expected Output	Comments
add_mult_positive	Add "a" with mult=3	Count = 3	ISP: positive mult
add_mult_zero	Add "a" with mult=0	"a" does not exist	BVA: 0 boundary

Test Case	Input	Expected Output	Comments
add_mult_negative	Add "a" with mult=-5	"a" does not exist	BVA: negative boundary
add_mult_nullKey	Add null with mult=2	Count for null = 2	Edge: null key

Coverage:

- mult > 0 branch
- Ignored branch

3. **Function: addAll(Multiset<T>)**

Partitions (ISP)	BVA
null input, empty input, normal input, merge existing, duplicates, null key	size = 0, 1, >1

Test Cases:

Test Case	Input	Expected Output	Comments
addAll_multiset_normal	**ms2 contains "a"^2, ms1 empty**	**ms1 "a" = 2**	**ISP: normal copy**
addAll_multiset_mergeExisting	**ms1 "a"^2, ms2 "a"^3**	**ms1 "a" = 5**	**ISP: merge existing**
addAll_multiset_emptyOther	**ms2 empty**	**ms1 remains empty**	**BVA: 0-size boundary**
addAll_multiset_null	**other=null**	**ms1 remains empty**	**ISP: null input**
addAll_multiset_withNullKey	**ms2 null^2**	**ms1 null^2**	**Edge: null key**

Coverage:

- If(other != null)
- Loop execution / skip

4. **addAll(Collection<T>)**

Partitions (ISP)	BVA
null collection, empty collection, collection with values	size = 0, 1, >1

Test Cases:

Test Case	Input	Expected Output	Comments
addAll_collection_normal	**Collection: ["a","b","a"]**	**"a"=2, "b"=1**	**ISP: normal collection**
addAll_collection_empty	**Collection: []**	**empty multiset**	**BVA: size=0**
addAll_collection_null	**Collection: null**	**empty multiset**	**ISP: empty**
addAll_collection_duplicates	**Collection: ["x","x","x"]**	**"x"=3**	**ISP: repeated values**
addAll_collection_withNull	**Collection: [null,null]**	**null=2**	**ISP: null value**

Coverage:

- Loop executed
- Loop not executed

5. **Function: addAll(T[] values)**

Partitions (ISP)	BVA
Null, empty, normal, duplicates	length = 0, 1, >1

Test Cases:

Test Case	Input	Expected Output	Comments
addAll_array_normal	**Array: ["a","b","a"]**	**"a"=2, "b"=1**	**ISP: normal array**

Test Case	Input	Expected Output	Comments
addAll_array_empty	Array: []	empty multiset	BVA: length=0
addAll_array_null	Array: null	empty multiset	BAD case
addAll_array_duplicates	Array: ["x","x","x"]	"x"=3	ISP: repeated values
addAll_array_withNull	Array: [null,null]	null=2	ISP: null element

Coverage:

- Loop executed

6. **Function: remove(Object key)**

Partitions (ISP)	BVA
exists (>1), exists (=1), not exists, null	Equal 1 Greater than 1

Test Cases:

Test Case	Input	Expected Output	Comments
remove_decreaseCount	Add "a"^5, remove "a"	"a"=4	BVA: count>1
remove_lastElement	Add "a"^1, remove "a"	"a" removed	BVA: count=1
remove_notExisting	Remove "x"	null	ISP: non-existing
remove_nullKey	Add null, remove null	null removed	ISP: null key

Coverage:

- All branches executed

7. **Function: remove(T key, int mult)**

Partitions (ISP)	BVA
partial removal , over-removal zero ,not exist	zero removal, exact removal(equal)

Test Cases:

Test Case	Input	Expected Output	Comments
remove_mult_normal	"a"^5, remove "a"^2	"a"=3	ISP: remove multiple
remove_mult_exact	"a"^3, remove "a"^3	"a" removed	BVA: exact removal
remove_mult_moreThanExists	"a"^2, remove "a"^10	"a" removed	ISP: over-removal
remove_mult_zero	"a"^2, remove "a"^0	"a"=2	BVA: zero removal
remove_mult_notExist	remove "x"^3	return 0	ISP: non-existing

Coverage:

- Handle Removing cases

8. **Function: removeAll(T key)**

Partitions (ISP)	BVA
Not-exist, exist, null	Eual removes

Test Cases:

Test Case	Input	Expected Output	Comments
removeAll_existing	"a"^4	returns 4, "a" removed	ISP:Normal remove
removeAll_notExisting	"x"	returns 0	ISP: remove not exist value
removeAll_exactOne	"a"^1	returns 1, "a" removed	BVA: exact removal
removeAll_afterMultipleOperations	"a"^2, "b"^3, remove "a"	returns 2, "a" removed, "b"=3	Multiple keys

Coverage:

- Handle Removing cases

9. **Function: intersect(Multiset<T>)**

Partitions (ISP)	BVA
Common elements, no Common elements, other = null	multiplicity = 1 multiplicity > 1

Test Cases:

Test Case	Input	Expected Output	Comments
intersect_basicHappy	a "x"^3, b "x"^2	"x"=2	BVA: Common
intersect_noCommonElements	a "a", b "b"	empty	ISP: no Common
intersect_otherIsNull	other=null	empty	ISP: null input
intersect_boundaryCountOne	a "a"^1, b "a"^5	"a"=1	BVA boundary

Coverage:

- All conditions (if/else branches) executed

10. **Function: totalCount()**

Partitions (ISP)	BVA
Empty, null, multiple inputs	0, 1 (single element), large elements

Test Cases:

Test Case	Input	Expected Output	Comments
totalCount_normal	"a"^2, "b"^3	5	ISP: multiple elements
totalCount_emptySet	empty	0	BVA: empty
totalCount_boundaryOne	"a"	1	BVA: single element
totalCount_largeValues	"a"^1000	1000	BVA: large

Test Case	Input	Expected Output	Comments
totalCount_withNullKey	null^3	3	ISP: null key

Coverage:

- Loop entered / not entered

11. Function: toList()

Partitions (ISP)	BVA
Null key, multiple , single	multiplicity = 1, >1

Test Cases:

Test Case	Input	Expected Output	Comments
toList_normal	"a"^2	list size=2, contains "a"	ISP: normal
toList_multipleElements	"a"^2, "b"	list size=3	ISP: multiple
toList_empty	empty	empty list	ISP: empty
toList_boundaryOne	"x"	list size=1	ISP: single, BVA: =1
toList_largeMultiplicity	"a"^100	list size=100	BVA: > 1
toList_withNullKey	null^2	list size=2, contains null	ISP: null key

Coverage:

- Nested loops

12. Function: toString()

Partitions (ISP)	BVA
empty set, single element, multiple elements, null key	multiplicity = 1, >1 large data

Test Cases:

Test Case	Input	Expected Output	Comments
toString_emptySet	empty	"{}"	ISP: empty set

Test Case	Input	Expected Output	Comments
toString_singleElement	"a"	"{a}"	ISP: Single element
toString_multipleElements	"a",b	"{a, b}" unordered	ISP: Multiple elements
toString_multiplicityGreaterThanOne	"a"^3	contains "a^3"	BVA: >1
toString_nullKey	null^2	contains "null^2"	ISP: null key
toString_largeData	"x"^50	contains "x^50"	BVA: large data

Coverage:

- If(size>0) and else branch
- Multiplicity condition

13. Function: equals(Object o)

Partitions (ISP)	BVA
equal sets, different size, different multiplicity, null object, different type	multiplicity = 1 vs >1

Test Cases:

Test Case	Input	Expected Output	Comments
equals_sameContentDifferentOrder	a: "a",b; b:"b",a	true	ISP
equals_sameElementsSameMultiplicity	a,b "a"^3	true	BVA
equals_differentMultiplicity	a "a"^2, b "a"^1	false	ISP
equals_differentSize	a "a", b "a","b"	false	ISP
equals_nullObject	null	false	ISP
equals_differentType	string object	false	ISP

Test Case	Input	Expected Output	Comments
equals_withNullKey	null^2	true	ISP

Coverage:

- All branches (if conditions)

14. Function: hashCode()

Test Case	Input	Expected Output	Comments
hashCode_shouldThrow	empty or any multiset	throws IllegalStateException	Designedexception

Coverage:

- Exception path